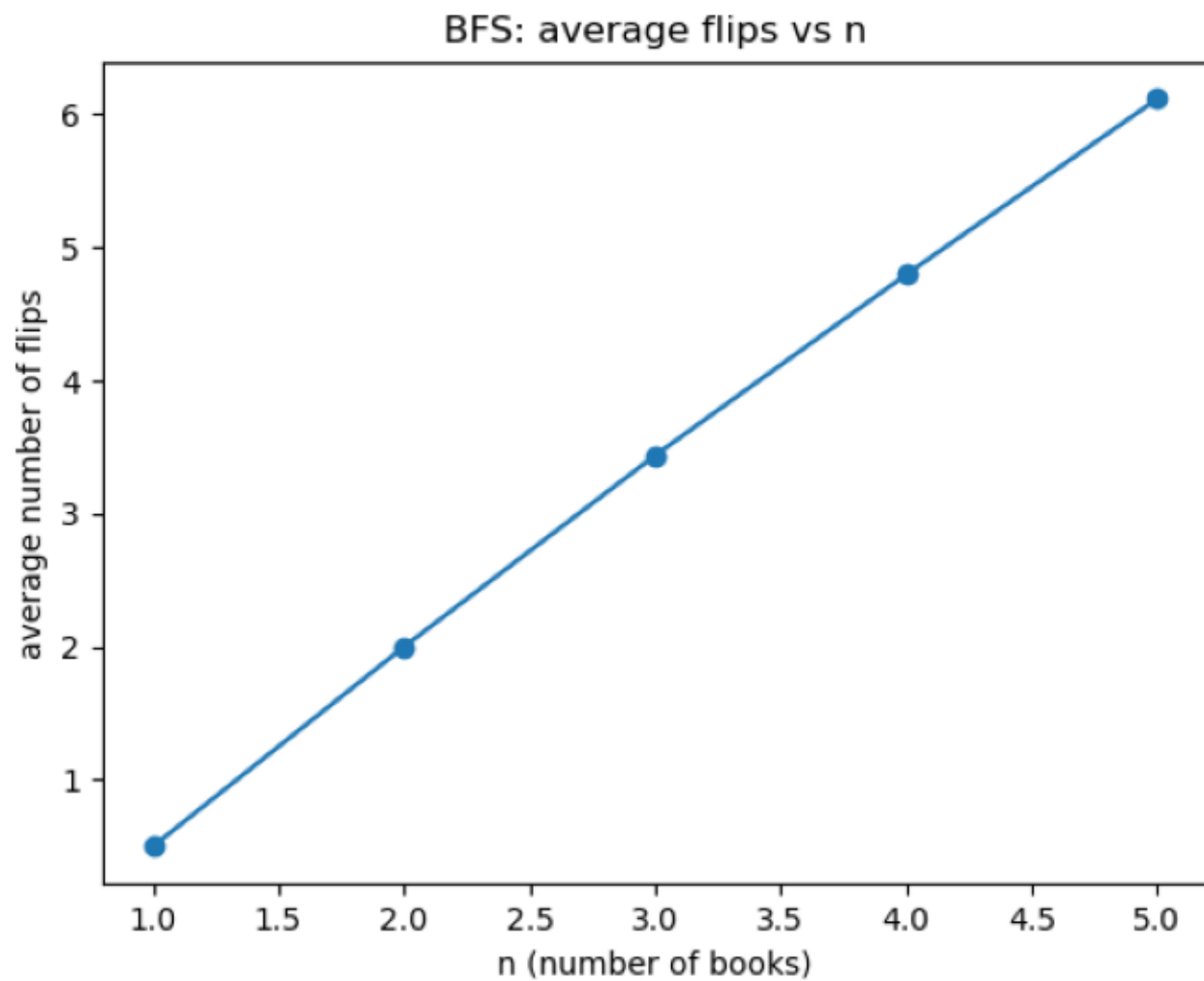
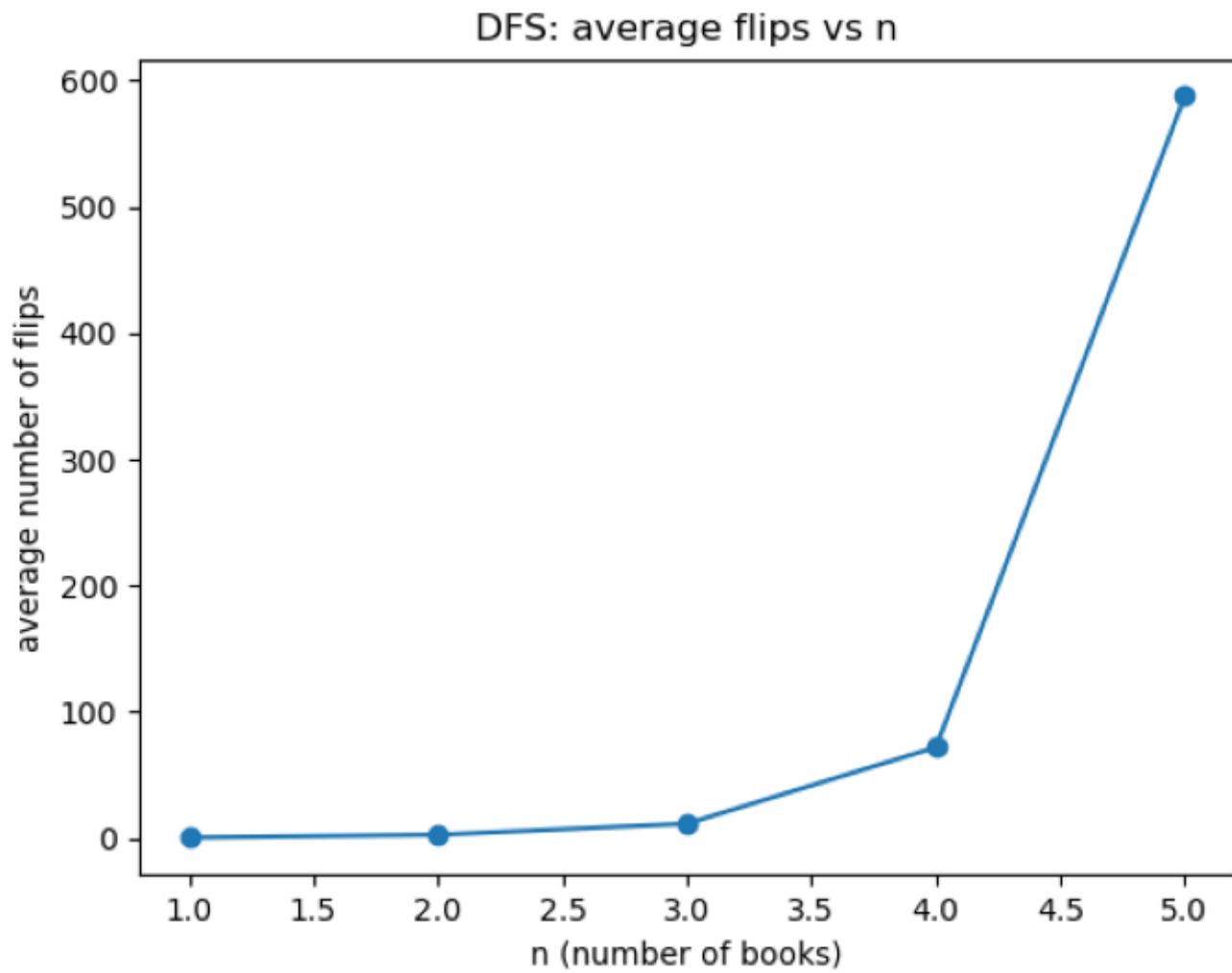


BFS: Average Number of Flips vs n



DFS: Average Number of Flips vs n



```
def average_flips_and_nodes(search_function_with_count, n_values):
    # exhaustive average, per n, of flips and nodes traversed
    average_flips_list = []
    average_nodes_list = []

    for n in n_values:
        total_flips = 0
        total_nodes = 0
        all_stacks = all_initial_stacks(n) # every  $2^n * n!$  stack

        for order, orientations in all_stacks:
            one_stack = TextbookStack(order, orientations)
            flip_sequence, node_count = search_function_with_count(one_stack)
            total_flips = total_flips + len(flip_sequence)
            total_nodes = total_nodes + node_count

        num_stacks = len(all_stacks)
        average_flips_list.append(total_flips / num_stacks)
        average_nodes_list.append(total_nodes / num_stacks)

    return average_flips_list, average_nodes_list
```

Lab 1 Extra Credit: Analysis and Discussion

Method: for each $n = 1..5$, I generated every one of the $2^n * n!$ possible initial stacks (not a random sample), ran BFS and DFS on each one, and averaged the flips and the nodes traversed across all of them. "Nodes traversed" = how many distinct states got visited before the sorted state was found.

Results:

n	stacks	BFS flips	BFS nodes	DFS flips	DFS nodes
1	2	0.500	1.500	0.500	1.500
2	8	2.000	4.500	2.750	4.500
3	48	3.438	24.500	11.521	24.500
4	384	4.797	192.500	72.055	192.500
5	3840	6.117	1920.500	587.697	1920.500

Flips: BFS barely moves (0.5 to about 6 flips from $n=1$ to $n=5$), while DFS blows up fast -- by $n=5$ it needs almost 100x more flips than BFS on average (587.7 vs 6.1). That gap matches what I'd expect: BFS explores level by level so the first time it reaches a sorted stack, that path is already the shortest one possible. DFS just dives down whichever branch it tries first (position 1, then 2, then 3, ...) with no sense of how close that branch is to the goal, so it can wander through a big chunk of the state space before it happens to land on the answer, and that chunk gets a lot bigger as n grows and the state space ($n!$ times 2^n) grows.

Nodes: this is the part that actually surprised me. BFS and DFS have the exact same average node count at every single n , down to three decimal places. My first thought was that I'd written a bug and both functions were secretly running the same search. But I checked it directly ($n=3$ example): if you list out the node counts for all 48 starting stacks, BFS gives you every value from 1 to 48 exactly once, and so does DFS -- just in a different order depending on the stack. Since both lists are just some rearrangement of 1, 2, ..., 48, they add up to the same total no matter what order they come in, so the average is forced to be $(48+1)/2 = 24.5$ either way. The same thing happens at every n : the average number of nodes always comes out to $(2^n * n! + 1) / 2$, and that formula doesn't care whether the search is BFS or DFS. So averaging node counts over every possible start turns out to be a bad way to compare the two algorithms -- it's actually measuring a property of the whole state graph, not of the search strategy. The flip counts above are where the real BFS vs. DFS difference actually shows up.

Overall takeaway: BFS finds the optimal (shortest) solution every time and its cost stays small and predictable as n grows. DFS's solution length is essentially a gamble that gets worse and worse as n grows, even though, on average over all starting positions, it doesn't actually "see" more of the graph than BFS does before stopping.